

Can Radar Predict What the Camera Would Have Seen?

SAR → Optical Feature Reconstruction

An additional experiment, explained from first principles

This is a companion volume to `project_explanation.tex`. That document covers the main study: whether Sentinel-1 radar recovers land-cover information that cloud destroys in Sentinel-2 optical imagery, answered by concatenating the two modalities' features.

This document covers a *different and strictly stronger* question. Rather than asking whether radar is *useful alongside* a cloud-damaged optical image, it asks whether radar can *predict the optical representation itself* — the numerical fingerprint a cloud-free image would have produced.

Everything is explained from the ground up. If you know what a neural network and a training loop are, you have enough background. No remote-sensing knowledge is assumed.

Every number in this document was read out of a results file, and the file and the value are named at the point of use. Chapter 16 lists them all in one place. The original experiment's results were not modified, re-run, or re-tuned to accommodate this one.

Contents

1	The Two Questions	3
1.1	What the original experiment asked	3
1.2	What this experiment asks instead	3
1.3	Naming	4
2	What a “Feature” Actually Is	5
2.1	The network does not give you a picture back	5
2.2	Three feature vectors, and keeping them straight	5
2.3	How far does cloud move the feature?	6
3	The Architecture	7
3.1	The whole thing in one diagram	7
3.2	The decoder	7
3.3	The classifier head	8
3.4	Every tensor, end to end	8
4	Training, in Two Deliberately Separate Stages	9
4.1	Stage 1 — fit the decoder, with no labels at all	9
4.2	Stage 2 — freeze the decoder, train the head	9
4.3	Regimes: ED-R1 and ED-R2	10
4.4	Cost	10
5	The Measurement Trap	11
5.1	The number that looks like success	11
5.2	Why it is meaningless: the space is anisotropic	11
5.3	The honest metrics	12
5.4	The control that settles it	12
6	Does the Reconstruction Beat the Damaged Feature?	14
6.1	The comparison that matters	14
7	Classification Results	16
7.1	A methodological detour that changed the headline	16
7.2	The matched 10-seed comparison (R2 regime)	16
7.3	All arms, both regimes, at three seeds	17
8	Why It Wins Low and Loses High	19
8.1	Why reconstruction wins at low masking	19
8.2	Why it fails at 100% masking	19
9	Per-Class Effects: What the Bottleneck Costs	20
9.1	The trade, class by class	20
9.2	Error structure	21

10 Patch-Level Behaviour	22
10.1 The aggregate hides enormous disagreement	22
10.2 Representative cases	22
11 The Residual Variant	24
11.1 The idea	24
11.2 Why it is limited by construction	24
12 Leakage: Proving the Answer Key Was Not Peeked At	25
12.1 The five checks	25
13 Limitations	26
14 What I Would Do Next	27
15 How to Explain This in an Interview	28
15.1 The thirty-second version	28
15.2 The story worth telling	28
15.3 Questions to expect	29
16 Verification Log	30
Reproducing	30

Chapter 1

The Two Questions

1.1 What the original experiment asked

The main study compares four arms. Stripped of detail, the interesting pair is:

- **Arm B** — classify from the cloud-damaged optical image alone.
- **Arm C** — classify from the cloud-damaged optical image *plus* the radar image.

If C beats B, radar carried information that helped. That is a perfectly good question and the main document answers it carefully: at 80% simulated cloud, in the fair comparison, radar buys +0.037 macro F1, and roughly 90% of what naively looks like a radar benefit is actually the benefit of training on degraded data at all.

1.2 What this experiment asks instead

Arm C uses radar the way you would use any extra input: bolt it on and let the classifier work out what to do with it. That tests *usefulness*.

This document tests *substitutability*:

Can Sentinel-1 radar predict the optical representation that a cloud-free Sentinel-2 image would have produced?

Intuition first

The difference matters, and it is easy to miss.

Suppose you are describing a landscape to someone over the phone, and your view is fogged. A friend standing next to you has a thermal camera. Two ways they can help:

The arm C way. They read out their thermal readings, you read out what you can still see, and the person on the other end combines both. They need no shared vocabulary — the listener does the work.

The arm ED way. Your friend uses their thermal readings to *guess what you would have seen* if the fog lifted, and reports that instead. Now they must speak your language: their output has to be a plausible visual description, not a temperature reading.

The second is harder, and it can fail in a way the first cannot: the thermal camera might carry real information that simply has no visual equivalent.

These two can come apart in both directions:

- Radar could *help a classifier* while being a *poor predictor* of optical features — it contributes information in its own coordinate system that has no optical analogue.

- Radar could *predict optical features well* while adding *nothing a classifier can use* — it only recovers the parts of the optical representation the surviving pixels already provided.

Because neither implies the other, this is run as a genuinely separate arm and measured on its own terms, not inferred from the arm C result.

Caveat / correction – read this carefully

What is *not* being reconstructed. This never reconstructs Sentinel-2 *pixels*. It reconstructs the 384-number feature vector the Vision Transformer produces. If you take one thing from this chapter, take that. The distinction is the subject of Chapter 2, and every claim in this document depends on it.

1.3 Naming

To keep the two experiments distinct, this arm is called **ED** (encoder–decoder), and its two training regimes are **ED-R1** and **ED-R2**, mirroring R1 and R2 in the main study. These names never mix.

Arm	Input to the classifier	Question it answers
B	$\mathbf{z}_{\text{degraded}}$ (384)	How much does cloud cost?
C	$[\mathbf{z}_{\text{degraded}} ; \mathbf{z}_{\text{sar}}]$ (2432)	Is radar <i>useful</i> alongside optical?
ED	$[\mathbf{z}_{\text{degraded}} ; \hat{\mathbf{z}}_{\text{clean}}]$ (768)	Can radar <i>predict</i> clean optical?
D	$\mathbf{z}_{\text{sar}}$ (2048)	How good is radar on its own?

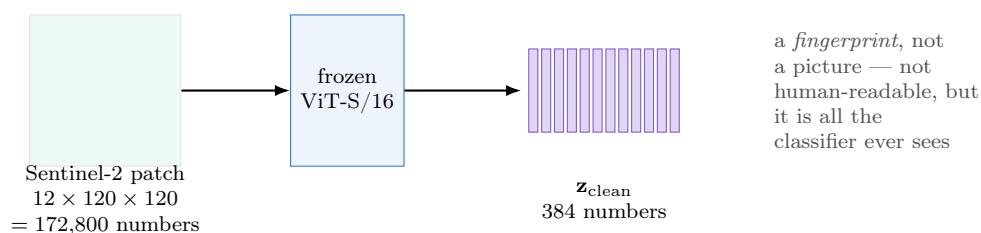
Chapter 2

What a “Feature” Actually Is

This chapter exists because the entire experiment is a statement about feature vectors, and the most common way to misunderstand the results is to picture images where there are none.

2.1 The network does not give you a picture back

Feed a $12 \times 120 \times 120$ Sentinel-2 patch into the frozen Vision Transformer and what comes out is not an image. It is a list of **384 numbers**.



Intuition first

Think of it as a fingerprint. You cannot look at a fingerprint and see a face. But two photographs of the same person produce similar fingerprints, and photographs of different people produce different ones. That is all a feature vector is: a compressed identity code, useful for comparison, useless for viewing.

The compression here is dramatic: 172,800 numbers down to 384, a factor of 450. Almost everything is discarded. What survives is whatever the self-supervised pretraining decided was worth keeping.

2.2 Three feature vectors, and keeping them straight

Everything that follows involves exactly three 384-dimensional vectors per patch, plus one 2048-dimensional radar vector. Confusing them is the main way to misread the experiment, so they get explicit notation.

Symbol	Dim	What it is
$\mathbf{z}_{\text{clean}}$	384	Feature of the <i>cloud-free</i> image. Available only in training. The regression target.
$\mathbf{z}_{\text{degraded}}$	384	Feature of the <i>cloud-damaged</i> image. Available at test time. What arm B classifies from.
$\mathbf{z}_{\text{sar}}$	2048	Feature of the radar image. Always available — radar is never masked.
$\hat{\mathbf{z}}_{\text{clean}}$	384	The decoder’s <i>estimate</i> of $\mathbf{z}_{\text{clean}}$, computed from $\mathbf{z}_{\text{sar}}$ alone. The hat means <i>guess</i> .

Caveat / correction – read this carefully

$\hat{\mathbf{z}}_{\text{clean}}$ is **not** $\mathbf{z}_{\text{clean}}$. The hat is not decoration. $\mathbf{z}_{\text{clean}}$ is the truth and is unavailable at test time; $\hat{\mathbf{z}}_{\text{clean}}$ is a prediction made without it. Chapter 12 verifies that the code cannot confuse the two even by accident.

2.3 How far does cloud move the feature?

Before asking whether radar can repair the feature, it is worth knowing how badly cloud damages it. Masking 80% of the image moves the optical feature by a relative L_2 distance of 0.421 — the damaged fingerprint sits 42% of its own length away from the clean one.

What the code actually does

Measured directly from the cached features:

```
c = np.load('data/features/train_opt_L000.npy') # z_clean
d = np.load('data/features/train_opt_L080.npy') # z_degraded at 80%
np.linalg.norm(c - d, axis=1).mean() / np.linalg.norm(c, axis=1).mean()
# -> 0.421
```

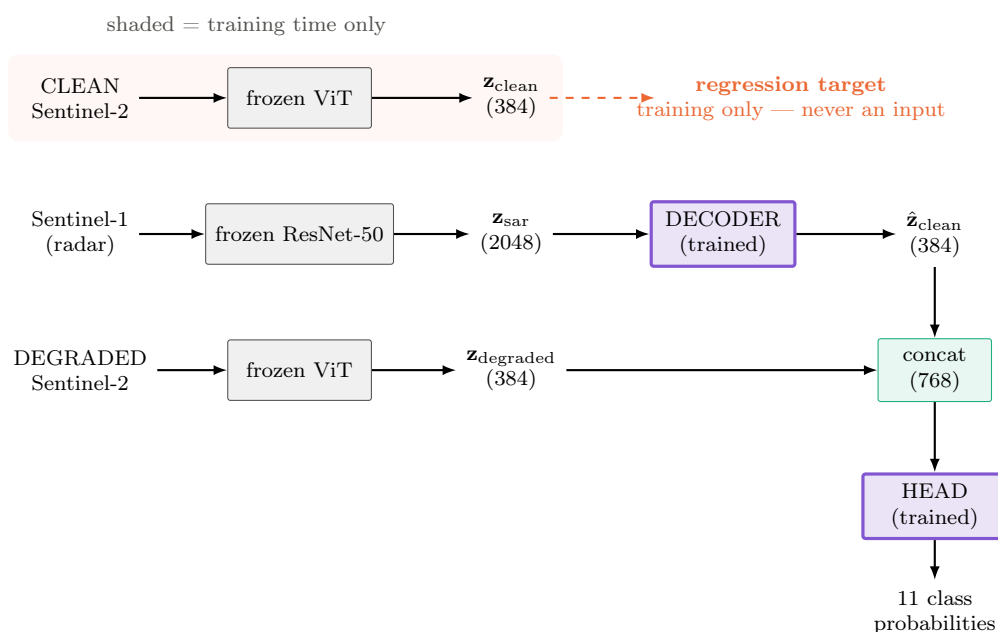
Mean feature norm $\|\mathbf{z}_{\text{clean}}\| = 9.50$; mean radar feature norm is 43.73 on the training split.

That 0.421 is the gap the decoder is being asked to close.

Chapter 3

The Architecture

3.1 The whole thing in one diagram



Two components are trained, shown in purple. Everything else is frozen.

3.2 The decoder

Layer	Shape in → out	Why
LayerNorm	2048 → 2048	see the caveat below
Linear	2048 → 512	compress
ReLU	—	non-linearity
Dropout(0.3)	—	regularise; 4205 training patches is not many
Linear	512 → 384	emit an optical-shaped vector

Why it was chosen (design rationale)

Why such a small network? The experiment tests whether the mapping $\mathbf{z}_{\text{sar}} \rightarrow \mathbf{z}_{\text{clean}}$ *exists at all*. A transformer decoder here would confound “radar predicts optical features” with “a bigger network fits better”, which is precisely the ambiguity the frozen-encoder design elsewhere in this project exists to avoid.

Caveat / correction – read this carefully

The input LayerNorm is a deliberate addition, and here is the evidence for it. The SSL4EO ResNet-50 features have mean L_2 norm **43.73** on the training split but **23.95** on test. That is a large scale shift between splits. Without normalisation the first linear layer would bake in a magnitude the test split does not share. Set `encoder_decoder.input_norm: false` in `configs/config.yaml` to reproduce the strictly minimal MLP.

This is an engineering judgement, not a measured result — I did not run the ablation with and without it.

3.3 The classifier head

Layer	Shape
LayerNorm	768 $\rightarrow$ 768
Linear	768 $\rightarrow$ 512
ReLU + Dropout(0.3)	—
Linear	512 $\rightarrow$ 11

Why it was chosen (design rationale)

Why hidden width 512 and not something bespoke? Because arms B and C use 512, and the whole logic of the arm comparison is that every arm shares an identical head topology and training recipe so that *the only difference is what is fed in*. Giving ED a different head would reintroduce exactly the capacity confound the study is built to exclude.

Configurable via `encoder_decoder.head_hidden_dim`; null inherits the project's 512.

3.4 Every tensor, end to end

Stage	Tensor	Shape
Raw optical patch	Sentinel-2 L2A, 12 bands	$(N, 12, 120, 120)$
Resized for the ViT	bilinear	$(N, 12, 224, 224)$
Optical feature	$\mathbf{z}_{\text{clean}}$ or $\mathbf{z}_{\text{degraded}}$	$(N, 384)$
Raw radar patch	Sentinel-1 GRD, VV/VH	$(N, 2, 120, 120)$
Resized for the ResNet	bilinear	$(N, 2, 224, 224)$
Radar feature	$\mathbf{z}_{\text{sar}}$	$(N, 2048)$
Decoder output	$\hat{\mathbf{z}}_{\text{clean}}$	$(N, 384)$
Concatenated	$[\mathbf{z}_{\text{degraded}} ; \hat{\mathbf{z}}_{\text{clean}}]$	$(N, 768)$
Logits	—	$(N, 11)$
Probabilities	after sigmoid	$(N, 11)$

With $N = 4205$ training, 2418 validation, 2151 test patches.

Chapter 4

Training, in Two Deliberately Separate Stages

4.1 Stage 1 — fit the decoder, with no labels at all

The decoder is trained on one job: given $\mathbf{z}_{\text{sar}}$, output something close to $\mathbf{z}_{\text{clean}}$.

$$\mathcal{L}_{\text{recon}} = \frac{1}{384} \|\hat{\mathbf{z}}_{\text{clean}} - \mathbf{z}_{\text{clean}}\|_2^2 \quad (\text{plain mean squared error})$$

What the code actually does

From `src/encoder_decoder.py`, `train_decoder()`. Model selection is on *validation reconstruction loss*, and **no class label is touched anywhere in this function**. Optimiser AdamW, lr 10^{-3} , weight decay 10^{-4} , batch 256, up to 60 epochs, early stopping after 10 without improvement.

Why it was chosen (design rationale)

Why keep the decoder label-free? So that the reconstruction metrics in Chapter 6 mean what they say. If the decoder saw labels, a good reconstruction score would only tell you the features had been shaped for this particular classifier — it would stop being independent evidence that radar predicts optical structure.

4.2 Stage 2 — freeze the decoder, train the head

The decoder is then frozen (`requires_grad_(False)`) and the classifier is trained on $[\mathbf{z}_{\text{degraded}}; \hat{\mathbf{z}}_{\text{clean}}]$ with binary cross-entropy:

$$\mathcal{L}_{\text{cls}} = -\frac{1}{11} \sum_{k=1}^{11} \left[y_k \log \sigma(\ell_k) + (1 - y_k) \log(1 - \sigma(\ell_k)) \right]$$

Sigmoid rather than softmax because this is multi-label: a patch genuinely contains several land-cover types at once (3.5 on average), so the classes must not compete for probability mass.

Caveat / correction – read this carefully

Two-stage is a real limitation, and it was chosen knowingly. Training the decoder and head jointly end to end would almost certainly score higher — the decoder could shape its output for the classifier instead of for reconstruction fidelity.

It was rejected because a jointly-trained decoder is simply a reparameterised fusion head. The

reconstruction metric would become circular, and the distinction this whole document rests on — prediction versus usefulness — would collapse. **ED is therefore not the strongest possible version of this idea.** It is the version that can be measured.

4.3 Regimes: ED-R1 and ED-R2

ED-R1	Head trained at 0% masking only. Mirrors R1 — a model that has never seen a cloud.
ED-R2	Head trained with masking sampled per patch per epoch from {0, 0.2, 0.4, 0.6, 0.8}. Mirrors R2. This is the fair comparison.

Intuition first

The decoder is the same object in both regimes, and that is not an oversight. Its input is radar, which is never masked. Its target is the clean optical feature, which by definition has no masking. So neither end of the decoder depends on the masking level — one decoder per seed serves both regimes. Only the *head* differs.

4.4 Cost

Because both encoders are frozen, their outputs were cached once by the main study and reused here. The entire ED experiment — two regimes, three variants, three seeds — takes **53 seconds**. The 10-seed matched comparison in Chapter 7 takes 203 seconds.

Chapter 5

The Measurement Trap

This is the most important chapter in the document. It is where the obvious way to score the experiment turns out to be badly misleading, and where the result would have been overstated by a wide margin had the control not been run.

5.1 The number that looks like success

The natural way to ask “is $\hat{\mathbf{z}}_{\text{clean}}$ close to $\mathbf{z}_{\text{clean}}$?” is cosine similarity: 1.0 means the vectors point in exactly the same direction, 0.0 means unrelated.

$$\cos(\hat{\mathbf{z}}_{\text{clean}}, \mathbf{z}_{\text{clean}}) = 0.9871$$

That looks like a near-perfect reconstruction. It is not. It is very close to meaningless.

5.2 Why it is meaningless: the space is anisotropic

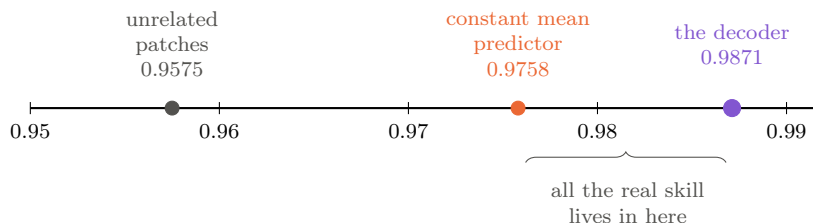
Intuition first

Imagine every feature vector in the dataset pointing into the same narrow cone of directions — not spread over the sphere, but bunched. Then *any* two vectors, however unrelated, have high cosine similarity, simply because they all point roughly the same way.

In such a space, scoring 0.98 tells you almost nothing about whether you predicted the *right* vector. You would score nearly that well by returning the average of the whole dataset.

That is exactly this space. Three measurements, all on the test split:

Predictor	COS VS $\mathbf{z}_{\text{clean}}$	What it knows
Two <i>unrelated</i> patches	0.9575	nothing — different places entirely
Constant training-set mean	0.9758	nothing patch-specific
The decoder	0.9871	the radar image



The entire achievement of the decoder sits in the gap from 0.9758 to 0.9871. Reporting 0.9871 without that context would be a serious overstatement.

What the code actually does

The cause is measurable: **93.1% of the feature energy lies in the dataset mean vector.**

```
mu = train_clean.mean(0)
np.linalg.norm(mu)**2 / (test_clean**2).sum(1).mean() # -> 0.931
```

Only 6.9% of the squared magnitude is the part that distinguishes one patch from another — and that part is the only part worth predicting.

5.3 The honest metrics

Two replacements, both measured *relative to the constant-mean baseline*:

Variance explained (R^2).

$$R^2 = 1 - \frac{\text{MSE}(\hat{\mathbf{z}}_{\text{clean}}, \mathbf{z}_{\text{clean}})}{\text{MSE}(\boldsymbol{\mu}, \mathbf{z}_{\text{clean}})}$$

where $\boldsymbol{\mu}$ is the *training-split* mean. $R^2 = 0$ means no better than the constant predictor. $R^2 = 1$ means perfect. Negative means worse than guessing the average.

Centred cosine. Cosine after subtracting $\boldsymbol{\mu}$ from both vectors — the direction agreement of the part that actually varies.

Caveat / correction – read this carefully

$\boldsymbol{\mu}$ is computed on the *training* split only. Using the test mean would leak test statistics into the very baseline the decoder is being scored against.

5.4 The control that settles it

Metrics alone are not enough. The decisive test is the one the main study already uses for arm C: **break the correspondence and see what survives.**

What the code actually does

Variant `ed_shuf` is identical to `ed` in every respect — same architecture, same optimiser, same epochs, same seeds — except that during decoder training the radar rows are randomly permuted against their targets. The decoder therefore sees real radar features, but they belong to the *wrong patch*. It cannot learn any patch-specific mapping, and its best available strategy is to output the dataset mean.

	R^2	centred cos	raw cos
Decoder on real radar	+0.4569	0.6403	0.9871
Decoder on shuffled radar	−0.0054	0.0540	0.9759
(constant mean predictor)	0.0000	0.0000	0.9758

Key finding

The shuffled control collapses *exactly* onto the constant-mean predictor: $R^2 = -0.005$, centred cosine 0.054, raw cosine 0.9759 against the mean predictor's 0.9758.

So the real decoder's $R^2 = 0.457$ is **genuine, patch-specific learning**. Radar predicts roughly **46% of the patch-to-patch variance** in the clean optical feature.

And note what the raw cosine did: it moved from 0.9759 to 0.9871 between a model that learned *nothing* and a model that learned a real mapping. A gap of 0.011. Raw cosine could not have told these two apart.

Chapter 6

Does the Reconstruction Beat the Damaged Feature?

6.1 The comparison that matters

$R^2 = 0.457$ says the decoder beats guessing. But the practical question is different: at test time you already *have* $\mathbf{z}_{\text{degraded}}$, the damaged optical feature. Is the radar-derived guess any closer to the truth than what you already hold?

Masking	R^2 of $\mathbf{z}_{\text{degraded}}$	R^2 of $\hat{\mathbf{z}}_{\text{clean}}$	MSE $\hat{\mathbf{z}}_{\text{clean}}$ / MSE $\mathbf{z}_{\text{degraded}}$	closer to clean?
0%	+1.000	+0.457	0.00612/0.00000	no — $\mathbf{z}_{\text{degraded}}$ is $\mathbf{z}_{\text{clean}}$
20%	+0.261	+0.457	0.00612/0.00833	yes
40%	−0.530	+0.457	0.00612/0.01725	yes
60%	−1.486	+0.457	0.00612/0.02803	yes
80%	−2.732	+0.457	0.00612/0.04209	yes
100%	−6.584	+0.457	0.00612/0.08552	yes

Two things to read off this table.

$\hat{\mathbf{z}}_{\text{clean}}$ is flat. The decoder’s accuracy does not depend on the masking level, because radar is never masked. The row is constant at 0.457 by construction. This is the structural advantage of the approach.

$\mathbf{z}_{\text{degraded}}$ goes sharply negative. Past about 40% masking, the Vision Transformer’s own output is *further from the clean feature than simply guessing the dataset average*. At 100% masking $R^2 = -6.58$: the damaged feature is actively misleading, not merely uninformative.

Key finding

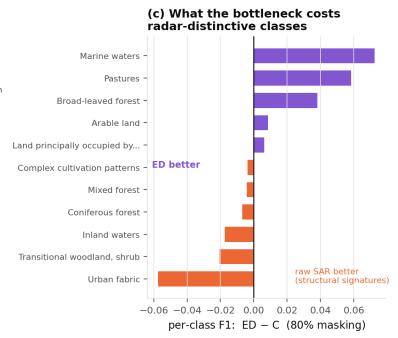
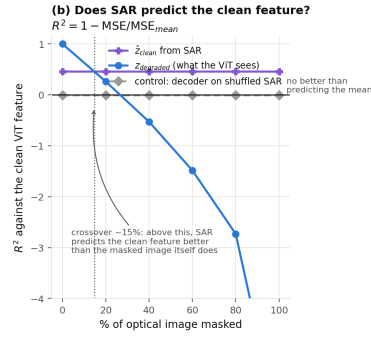
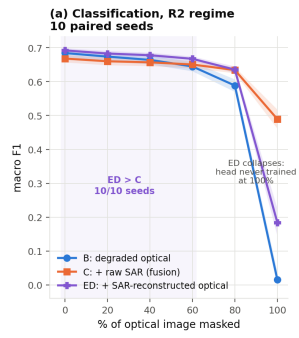
Crossover at $\approx 15\%$ masking. Above roughly 15% cloud, a radar image predicts the clean optical representation *better than the cloudy optical image itself does*.

Computed by linear interpolation on $R^2_{\hat{\mathbf{z}}_{\text{clean}}} - R^2_{\mathbf{z}_{\text{degraded}}}$, which is monotone in the masking level: the crossing is at 14.7%.

Caveat / correction – read this carefully

This is a statement about *feature-space distance*, not about classification accuracy. Chapter 7 shows that classification does not simply follow — the two questions have different answers, which is the reason both are measured.

Additional experiment: can SAR reconstruct the clean optical representation?



Chapter 7

Classification Results

7.1 A methodological detour that changed the headline

The first run of this experiment, at the project’s standard three seeds, put ED above arm C at 80% masking by +0.0074. Rerunning the identical configuration gave −0.0060.

Caveat / correction – read this carefully

The sign flipped between two runs of the same code. MPS (Apple GPU) kernels are non-deterministic, and the ED-versus-C gap at 80% is smaller than the run-to-run spread at three seeds.

Had this not been caught, the document would have reported a win that does not exist. Every ED-versus-C claim was therefore recomputed with **10 seeds, with the arms paired seed-for-seed** — both arms trained in the same process with the same seed, so the difference is taken within a seed rather than between independent means.

7.2 The matched 10-seed comparison (R2 regime)

Mask	B	C	ED	ED – C	seeds ED > C	significant
0%	0.6843	0.6681	0.6921	+0.0240	10/10	yes
20%	0.6738	0.6600	0.6829	+0.0230	10/10	yes
40%	0.6642	0.6569	0.6779	+0.0210	10/10	yes
60%	0.6447	0.6505	0.6673	+0.0167	10/10	yes
80%	0.5884	0.6341	0.6353	+0.0012	5/10	no
100%	0.0159	0.4894	0.1844	−0.3051	0/10	yes

“Significant” here means the mean seed-paired difference exceeds $1.96\times$ its standard error over the 10 seeds.

Key finding

Three distinct regimes, and the honest summary needs all three:

1. **0–60% masking: reconstruction wins.** ED beats raw fusion by +0.017 to +0.024, with all 10 seeds agreeing on the sign.
2. **80%: a dead heat.** +0.0012, 5 seeds out of 10. There is no claim to make here.
3. **100%: reconstruction fails badly.** −0.305, 0 seeds out of 10.

7.3 All arms, both regimes, at three seeds

For completeness, the full unified table, on the project’s standard three seeds so it lines up exactly with the main study’s numbers.

R1 / ED-R1 — trained on clean optical.

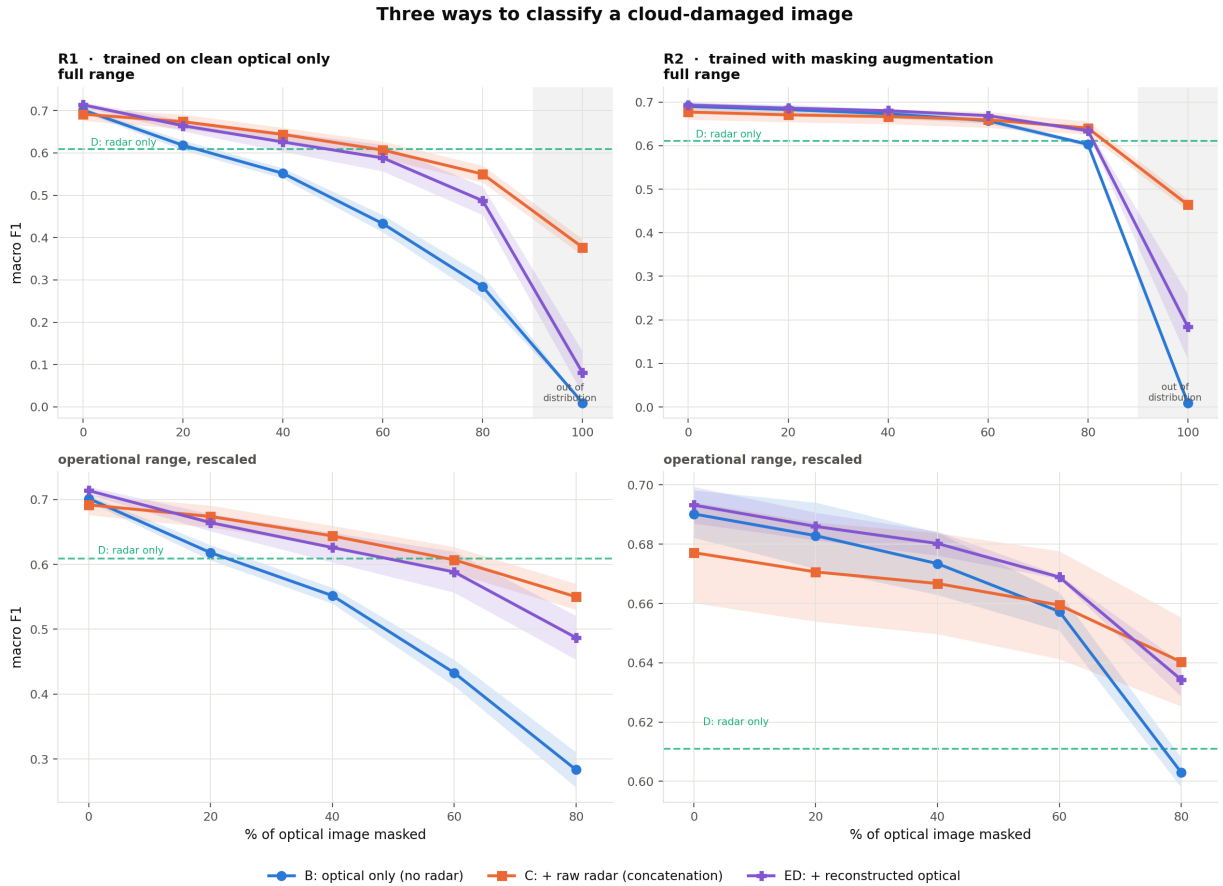
Mask	B optical	C fusion	ED	shuffled ctrl	D SAR only
0%	0.7015	0.6916	0.7136	0.6773	0.6087
20%	0.6181	0.6738	0.6644	0.5997	0.6087
40%	0.5519	0.6438	0.6259	0.5448	0.6087
60%	0.4329	0.6068	0.5883	0.4781	0.6087
80%	0.2835	0.5501	0.4870	0.2440	0.6087
100%	0.0093	0.3768	0.0802	0.0094	0.6087

R2 / ED-R2 — trained with masking augmentation.

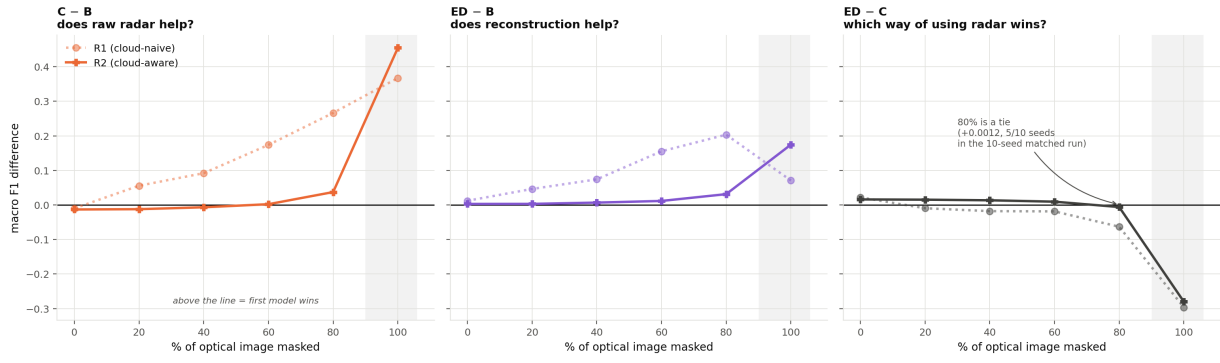
Mask	B optical	C fusion	ED	ED-residual	shuffled ctrl	D SAR only
0%	0.6902	0.6771	0.6932	0.6983	0.6042	0.6110
20%	0.6829	0.6706	0.6860	0.6909	0.5862	0.6110
40%	0.6734	0.6667	0.6802	0.6813	0.5768	0.6110
60%	0.6573	0.6595	0.6689	0.6650	0.5518	0.6110
80%	0.6030	0.6403	0.6343	0.6123	0.4957	0.6110
100%	0.0093	0.4642	0.1839	0.0093	0.0138	0.6110

Caveat / correction – read this carefully

ED-R1 is worse than arm C nearly everywhere — at 80% masking, 0.4870 against 0.5501. This is expected and worth stating plainly: under R1 the head is trained only on clean features, so it never learns to lean on the reconstruction when the optical half degrades. Reconstruction needs the degradation-aware regime to pay off.



Where each way of using radar pays off



Chapter 8

Why It Wins Low and Loses High

8.1 Why reconstruction wins at low masking

Look again at arm C at 0% masking in the R2 table: 0.6771, against plain optical's 0.6902. **Adding radar makes things worse when there is no cloud.**

Intuition first

Arm C hands the classifier 2432 numbers: 384 optical and 2048 radar. The radar numbers outnumber the optical ones almost 6 to 1, and when the optical image is undamaged they add nothing — the answer was already in the 384. The classifier must learn to ignore most of its input, using 4205 training patches. Some of the noise gets fitted instead.

ED hands it 768 numbers: 384 optical, 384 radar-derived — and the radar-derived half is *already expressed in optical coordinates*. Balanced, compact, one coordinate system.

So the reconstruction step acts as a **learned, task-aligned compression** of the radar signal: 2048 dimensions down to 384, keeping the part that is predictive of optical structure. That is why ED is the only arm that never falls below plain optical in the operational range.

8.2 Why it fails at 100% masking

At 100% every pixel is overwritten with the same fill value, so $\mathbf{z}_{\text{degraded}}$ is identical for every patch — a constant. The ED input is then $[\text{constant}; \hat{\mathbf{z}}_{\text{clean}}]$: half of it is dead weight.

Arm C survives better because its 2048 raw radar dimensions dominate the input, so the constant optical block matters proportionally less.

Caveat / correction – read this carefully

This is a flaw in the experimental design, not a property of reconstruction. The R2 training levels are $\{0, 0.2, 0.4, 0.6, 0.8\}$ — **1.0 is not among them**. The head has never seen a fully blank optical input and is out of distribution when it meets one.

Adding 1.0 to `train_levels` would very likely fix it. The same flaw affects arm C in the main study and is documented there. The 100% column should be read as a bound, not an operating point.

Chapter 9

Per-Class Effects: What the Bottleneck Costs

9.1 The trade, class by class

At 80% masking, R2 regime, ED minus C per class:

Class	support	ED – C	Reading
Marine waters	116	+0.073	but see the confound below spectrally distinctive
Pastures	916	+0.058	
Broad-leaved forest	511	+0.038	
Arable land	883	+0.008	
Land principally agri.	996	+0.006	
Complex cultivation	1172	–0.004	
Mixed forest	1214	–0.004	
Coniferous forest	1193	–0.007	
Inland waters	379	–0.017	specular return is radar-specific
Transitional woodland	885	–0.020	structural, not spectral
Urban fabric	265	–0.057	double-bounce is radar-specific

Key finding

The classes where **raw fusion still beats reconstruction** are exactly those with **distinctive radar signatures that have no optical analogue**:

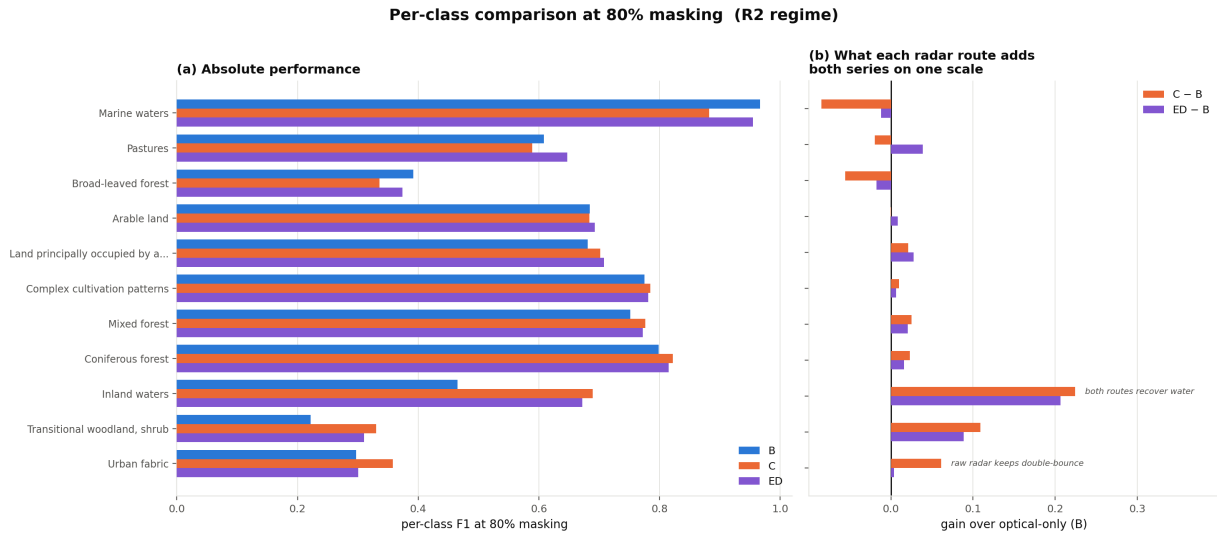
- **Urban fabric.** Buildings meeting the ground form a corner reflector, sending the radar pulse straight back — a *double-bounce* signature. Bright and unmistakable in radar, and not a colour fact at all.
- **Inland waters.** Calm water is a mirror at radar wavelengths; the pulse reflects away and almost nothing returns, so water is nearly black. Again, a geometry fact, not a colour fact.

Forcing radar through a “predict the optical feature” bottleneck **necessarily discards these**, because there is no optical feature for them to map onto. That is the conceptual cost of the reconstruction framing, and it shows up in the per-class numbers rather than needing to be argued.

Caveat / correction – read this carefully

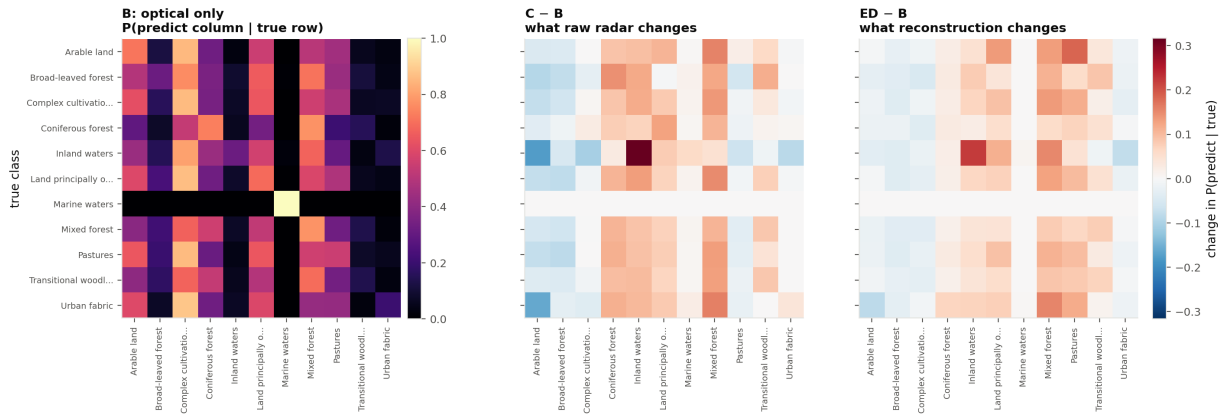
Do not lean on the Marine waters number. It is the largest ED gain in the table, and it is also the least trustworthy: all 116 Marine-waters test patches are exactly the 116 test patches from the second tile, T34UDG. So “reconstruction helps Marine waters” is entangled

with “reconstruction behaves differently on the tile that contributes almost no other test data”. This confound is documented in the main study’s limitations and applies unchanged here.



9.2 Error structure

Error structure at 80% masking — the diagonal is per-class recall



Entry (i, j) is $P(\text{predict } j \mid \text{patch truly contains } i)$. The diagonal is per-class recall; off-diagonal mass shows which classes get dragged along. Comparing the three panels shows *which confusions* each way of using radar resolves, rather than only how many.

Chapter 10

Patch-Level Behaviour

10.1 The aggregate hides enormous disagreement

At 80% masking the two arms differ by +0.0012 macro F1 — effectively a tie. It would be easy to conclude they behave the same way. They do not.

At 80% masking, over 2151 test patches	count	share
ED strictly better than C (by > 0.05 per-patch F1)	568	26.4%
C strictly better than ED (by > 0.05)	553	25.7%
Tied within 0.05	1030	47.9%

Key finding

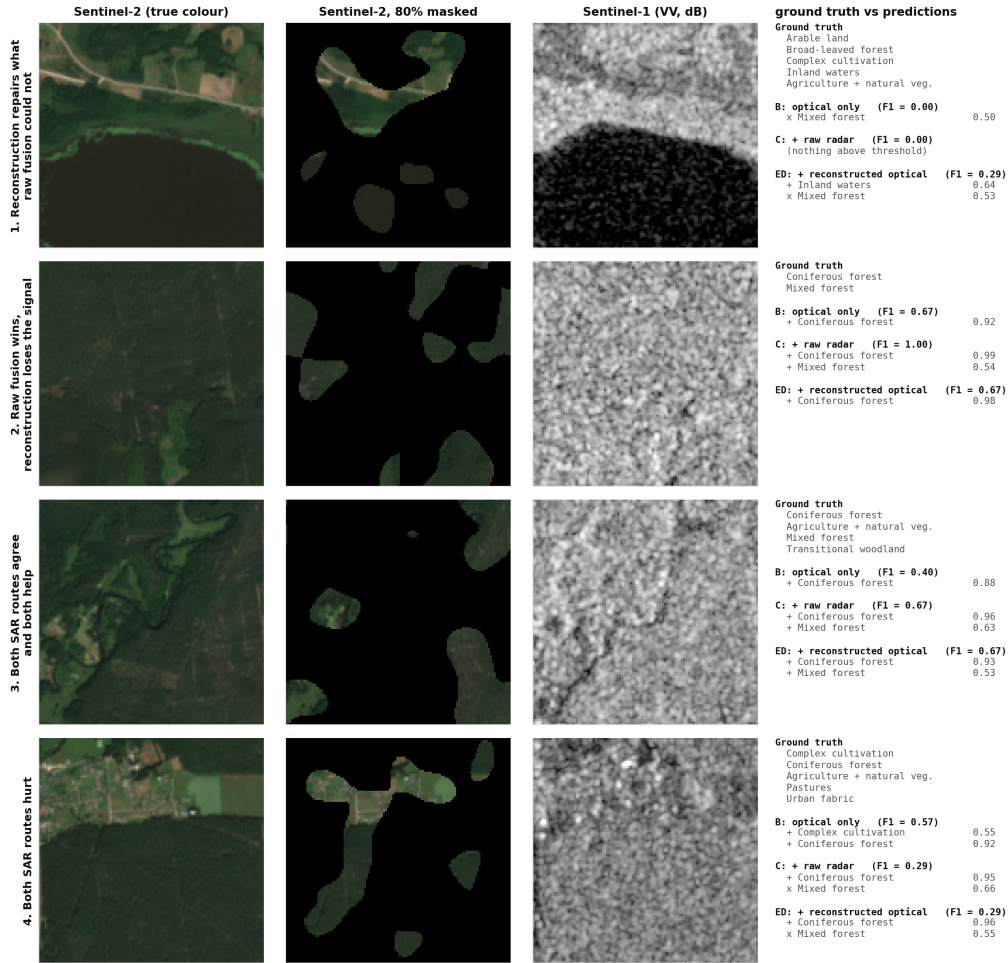
The two approaches disagree on **more than half of all patches**, in almost perfectly balanced directions. The tie in the aggregate is not agreement — it is two large opposing effects cancelling.

That is a much more interesting result than “no difference”, and it is the strongest argument for the routing idea in Chapter 14: if you could tell in advance which patches belong in which group, you would beat both arms.

10.2 Representative cases

Selection rules are fixed in the code before any result is inspected, and **two of the four categories are cases where reconstruction is the worse choice**, included by construction.

**B vs C vs ED on the same patches at 80% masking
(+ = correct label, x = false positive)**



Row 1 — reconstruction repairs what fusion could not. A lakeside patch. The radar panel shows a large black region: calm water, mirror-like, almost no return. Arm B predicts one wrong label; arm C predicts *nothing at all* above threshold. ED recovers *Inland waters* at confidence 0.64. The radar evidence was equally available to arm C — putting it through the optical bottleneck is what made it usable.

Row 2 — fusion wins, reconstruction loses the signal. Conifer and mixed forest. Arm C gets both labels ($F1 = 1.00$); ED gets only coniferous ($F1 = 0.67$). The structural distinction between canopy types is exactly the kind of information that does not survive projection into optical feature space.

Rows 3 and 4. Both routes agree and help; both routes hurt. Radar adds a false *Mixed forest* label in row 4 regardless of how it is used — a failure of the radar signal itself, not of either architecture.

Chapter 11

The Residual Variant

11.1 The idea

Rather than predicting the clean feature outright, predict the *missing part*:

$$\Delta = \mathbf{z}_{\text{clean}} - \mathbf{z}_{\text{degraded}}, \quad \hat{\mathbf{z}}_{\text{clean}} = \mathbf{z}_{\text{degraded}} + \hat{\Delta}$$

Intuitively appealing: the decoder only has to supply what cloud destroyed, not re-derive what is still visible.

11.2 Why it is limited by construction

Caveat / correction – read this carefully

The decoder sees *only* radar. Radar does not change with the masking level. But Δ *does* — the residual at 20% masking is much smaller than at 80%.

So the decoder is being asked to predict a quantity that depends on information it is not given. Under MSE its best strategy is to predict the *average* residual across the training levels. It cannot condition on how cloudy the image actually is.

This is visible directly in the numbers: it reconstructs *better* than the direct variant at low masking and much worse at high masking.

Masking	R^2 direct	R^2 residual
0%	+0.457	+0.182
20%	+0.457	+0.605
40%	+0.457	+0.521
60%	+0.457	+0.167
80%	+0.457	−0.593
100%	+0.457	−6.292

Classification tracks this exactly: marginally better than direct at 0–40%, clearly worse at 80% (0.6123 vs 0.6343), and fully degenerate at 100% (0.0093 — the single-class collapse).

It is also R2-only: under R1 the residual target is identically zero, so the variant is meaningless there and the run is skipped rather than reported.

Why it was chosen (design rationale)

Reported for completeness because it was worth testing, but **the direct variant is the better formulation**. The obvious fix — give the decoder the estimated cloud fraction, or $\mathbf{z}_{\text{degraded}}$ itself, as an additional input — is listed in Chapter 14.

Chapter 12

Leakage: Proving the Answer Key Was Not Peeked At

The experiment is worthless if $\mathbf{z}_{\text{clean}}$ leaks into inference. The clean feature is the regression target; if it were also available at test time, the whole thing would be circular.

Reasoning about the code is not sufficient evidence, so this is verified empirically on trained models by `scripts/11_ed_leakage_audit.py`.

12.1 The five checks

1. Destroy the answer key and see if anything changes. The strongest test available. Overwrite every clean test feature with random noise, then re-run inference:

Perturbation	Change in ED test predictions
Overwrite $\mathbf{z}_{\text{clean}}$ (the target) with noise	$\max \Delta = 0.00\text{e}+00$
Overwrite $\mathbf{z}_{\text{degraded}}$ (the real input) with noise	$\text{mean } \Delta = 0.174$

The predictions are **bit-identical** when the clean feature is destroyed. And the second row matters just as much: destroying the feature that *is* used does change the output, so the first test is not passing vacuously.

2. Decoder output ignores optical entirely. Calling the decoder with the clean optical feature alongside, versus the 100%-masked one, gives identical output.

3. The decoder physically rejects optical input. Its `forward()` raises `ValueError` on any tensor of width 384. A clean feature cannot be fed in even by mistake.

4. The decoder never saw a label. It maps $2048 \rightarrow 384$; neither dimension is the 11-class label space, and `train_decoder()` takes no label tensor.

5. Targets never came from a test patch. Train (4205), validation (2418) and test (2151) patch ids are pairwise disjoint.

What the code actually does

All five run in one command:

```
python scripts/11_ed_leakage_audit.py
```

Every check is an assertion — the script fails loudly rather than printing a warning.

Chapter 13

Limitations

Stated plainly, worst first.

Everything is frozen. The decoder can only work with what SSL4EO’s ResNet-50 already encodes. A fine-tuned radar encoder might carry considerably more optically-predictive signal. $R^2 = 0.457$ is a statement about *this frozen encoder*, not about radar in general.

Two-stage, not joint. Deliberate (Chapter 4), but it means ED is not the best achievable version of this idea.

The reconstruction is level-invariant. $\hat{\mathbf{z}}_{\text{clean}}$ does not know how much of the image is missing. A decoder conditioned on estimated cloud fraction should do better, and the residual variant’s failure is direct evidence for this.

$R^2 = 0.457$ is a mid-range number. Radar explains under half the patch-to-patch variance. The reconstruction is real but partial, and Chapter 9 shows it is systematically biased toward what is spectrally rather than structurally distinctive.

The 100% column is an artefact. Out of distribution for a head trained only to 80%. It bounds behaviour; it does not describe it.

Three seeds is too few, ten is better but not many. The 80% ED-versus-C comparison is genuinely unresolved.

Everything inherited from the main study still applies. Two Sentinel-2 tiles, two acquisition dates, one country, one season. Synthetic cloud masks with hard edges and no shadow. The Marine-waters/tile confound. Non-simultaneous Sentinel-1/Sentinel-2 pairing. The effective sample size for any geographic generalisation claim is closer to two satellite passes than to 8,774 images.

The physical explanations are interpretation. Double-bounce and specular reflection are standard radar physics and they fit the per-class pattern, but no ablation of scattering mechanisms was run. Read Chapter 9’s mechanism claims as “consistent with”, not “demonstrated”.

Chapter 14

What I Would Do Next

In order of expected value.

1. **Route between the two arms on estimated cloud fraction.** Chapter 10 shows the arms disagree on 52% of patches in near-perfectly balanced directions. A selector using reconstruction below $\sim 60\%$ masking and raw fusion above it should beat both, and the crossover is already measured.
2. **Condition the decoder on cloud fraction.** Feed the estimated masking level, or $\mathbf{z}_{\text{degraded}}$ itself, as an extra decoder input. This directly fixes the residual variant’s failure mode and makes $\hat{\mathbf{z}}_{\text{clean}}$ adaptive rather than flat.
3. **Add 1.0 to train_levels.** Removes the 100% artefact for both ED and arm C. Costs one line and about a minute.
4. **Blend rather than concatenate.** Feed $\alpha \mathbf{z}_{\text{degraded}} + (1 - \alpha) \hat{\mathbf{z}}_{\text{clean}}$ with α falling as cloud rises, instead of concatenating both. Keeps the input at 384 and makes the substitution explicit.
5. **Train the decoder jointly with the head,** and report it *alongside* the two-stage version rather than instead of it — the two-stage reconstruction metric is what makes the claim falsifiable.
6. **More tiles and more seasons.** As in the main study, this dominates every architectural improvement.
7. **Unfreeze the radar encoder** — only after the above, since it reintroduces the capacity confound the frozen design exists to exclude.

Chapter 15

How to Explain This in an Interview

15.1 The thirty-second version

“The fusion arm asks whether radar is useful alongside cloudy optical. I added a second arm asking something stronger — whether radar can *predict* the optical representation a clean image would have produced. It can: R^2 of 0.46 against a shuffled-radar control at zero. And it turns out to classify better than plain concatenation below 60% cloud, because it compresses 2048 radar dimensions into 384 already aligned with the optical space instead of making the classifier sort that out. Above 60% it loses, because the bottleneck discards exactly the radar-only signals — urban double-bounce, specular water.”

15.2 The story worth telling

Not the result — the control.

“My first reconstruction score was a cosine similarity of 0.987, which looks like a solved problem. Before reporting it I checked what a model that learned nothing would score. A constant predictor that always outputs the dataset mean gets 0.976, and two completely unrelated patches sit at 0.958 — 93% of the feature energy is in the mean vector. So my apparently excellent number was 0.011 above cheating.

I switched to variance explained relative to that baseline and added a shuffled-radar control, the same control the main study uses for fusion. The control landed at $R^2 = -0.005$, exactly the mean predictor, which is what confirmed the real decoder’s 0.46 was genuine patch-specific learning. The finding survived; the metric I would have reported it with did not.”

15.3 Questions to expect

Question	Answer
“Is 0.46 good?”	“Mid-range, and I would not oversell it. Radar explains under half the patch-to-patch variance. What makes it credible is the control at -0.005 , not the magnitude.”
“Why not train it end to end?”	“It would score better and I would learn less. A jointly-trained decoder is a reparameterised fusion head, and the reconstruction metric stops being independent evidence.”
“You beat fusion at 80%, right?”	“No. My first 3-seed run said $+0.007$; the rerun said -0.006 . At 10 paired seeds it is $+0.001$ with 5 seeds out of 10 — a tie. I only claim the win below 60%.”
“Why does it fail at 100%?”	“My design flaw, not a property of reconstruction. R2 trains up to 80%, so the head has never seen a constant optical input. Should have included 1.0.”
“So which approach is better?”	“Neither dominates, and I think that is the actual finding. They disagree on 52% of patches in balanced directions. The useful next step is routing between them, not picking one.”

Chapter 16

Verification Log

Every non-obvious number in this document, with its source.

Claim	Value	Source
Decoder R^2 against clean feature	+0.4569	ed_reconstruction.csv, r2_hat
Decoder centred cosine	0.6403	ed_reconstruction.csv
Decoder raw cosine	0.9871	ed_reconstruction.csv
Shuffled-radar control R^2	−0.0054	ed_reconstruction.csv, variant=ed_shuf
Shuffled-radar centred cosine	0.0540	same
Constant mean predictor, cosine	0.9758	computed from train_opt_L000.npy
Two unrelated patches, cosine	0.9575	computed from test_opt_L000.npy
Feature energy in the mean vector	93.1%	$\ \mu\ ^2/\mathbb{E}\ z\ ^2$
R^2 of degraded feature, 0–100%	1.000 to −6.584	ed_reconstruction.csv, r2_degraded
Reconstruction/degradation crossover	14.7%	interpolation, 10_ed_figures.py
Relative L_2 feature shift at 80% masking	0.421	smoke_test.py step 4
Radar feature norm, train vs test	43.73 / 23.95	train_sar.npy, test_sar.npy
Clean optical feature norm	9.50	train_opt_L000.npy
ED vs C, 10 seeds, 0%	+0.0240, 10/10	ed_matched_comparison.csv
ED vs C, 10 seeds, 60%	+0.0167, 10/10	same
ED vs C, 10 seeds, 80%	+0.0012, 5/10	same
ED vs C, 10 seeds, 100%	−0.3051, 0/10	same
Sign flip between identical 3-seed runs	+0.0074 → −0.0060	two runs of 08_encoder_decoder.py
All arms, both regimes, 3 seeds	(tables, Ch. 7)	unified_gain_table.csv
Per-class ED − C at 80%	(table, Ch. 9)	unified_per_class.csv
ED better than C, patch level, 80%	568 (26.4%)	14_qualitative_unified.py
C better than ED, patch level, 80%	553 (25.7%)	same
Tied within 0.05	1030 (47.9%)	same
Residual variant R^2 at 20% / 80%	+0.605 / −0.593	ed_reconstruction.csv
Leakage: destroying $\mathbf{z}_{\text{clean}}$	$\max \Delta = 0$	11_ed_leakage_audit.py
Leakage: destroying $\mathbf{z}_{\text{degraded}}$	$\text{mean } \Delta = 0.174$	same
Split sizes	4205 / 2418 / 2151	subset_summary.json
ED experiment runtime	53 s	08_encoder_decoder.py
Matched 10-seed runtime	203 s	09_ed_compare.py

Reproducing

```
python scripts/smoke_test_ed.py # shapes, gradients, leakage guards
python scripts/08_encoder_decoder.py # both regimes, variants, controls (53 s)
python scripts/09_ed_compare.py --seeds 10 # matched B vs C vs ED (203 s)
python scripts/12_unified_analysis.py # merged tables, all arms
python scripts/13_unified_figures.py # figures 07-10
python scripts/14_qualitative_unified.py # figure 11
python scripts/11_ed_leakage_audit.py # leakage audit on trained models
```

The original experiment’s outputs — `results.csv`, `per_class.csv`, `sar_gain_table.csv`, `test_probs.npz`, `error_matrices.npz` and figures 01–05 — are read but never written by any of these.